


Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

# Edytor schematów szydełkowych z automatycznym rozmieszczaniem symboli

Etap 1: Specyfikacja wymagań i określenie zakresu projektu

Autorzy:

Anna Babicka, Wiktor Murawski

Promotor:

dr inż. Paweł Kotowski

19.10.2026

# Spis treści

**1 Historia zmian dokumentu** **3**

**2 Opis rozwiązania i motywacja** **4**

**3 Słownik** **5**

3.1 Szydełkowanie . . . . . 5

3.2 Model i algorytmy . . . . . 5

3.3 Technologie i jakość . . . . . 6

**4 Zakres projektu: MVP i opcjonalne funkcjonalności** **7**

4.1 Minimum Viable Product (MVP – Passing Scope) . . . . . 7

4.2 Opcjonalne funkcjonalności . . . . . 7

**5 Wymagania funkcjonalne** **8**

**6 Mierzalne wymagania niefunkcjonalne (NFRs –Quantifiable Non-Functional Requirements)** **12**

**7 Zarządzanie ryzykiem – analiza SWOT** **14**

**8 Harmonogram i macierz RACI** **15**

8.1 Macierz odpowiedzialności (RACI) . . . . . 15

# 1 Historia zmian dokumentu

| Wersja | Data | Autor | Opis zmian |
|---|---|---|---|
| **1.0** | 18.10.2026 | Anna Babicka | *Pierwsza wersja dokumentu* |
| **1.1** | 19.10.2026 | Anna Babicka | *Wprowadzenie poprawek po konsultacji z promotorem* |

## 2 Opis rozwiązania i motywacja

Celem projektu jest zaprojektowanie, implementacja i ocena intuicyjnej aplikacji webowej do tworzenia schematów szydełkowych, która odciąża użytkownika od ręcznego rozmieszczania symboli dzięki reprezentacji wzorów jako grafów oczek i automatycznemu układowi uwzględniającemu geometrię oczek, umożliwiając szybkie i bezbłędne tworzenie czytelnych schematów wzorów.

# 3 Słownik

## 3.1 Szydełkowanie

| Termin | Definicja |
|---|---|
| **Oczko (ścieg, stitch)** | Podstawowa jednostka robótki, powstająca przez przeciągnięcie nici przez pętle na szydełku. |
| **Łańcuszek (ch)** | A Oczko niewbijane w robótkę. Podnosi poziom pracy i tworzy łuki. |
| **Oczko ścisłe (sl st)** | Oczko łączące dwa punkty bez zwiększania wysokości. Służy m.in. do zamykania okrążeń. |
| **Półsłupek (sc),<br>słupek półpodwójny (hdc),<br>słupek (dc),<br>słupek podwójny (tr)** | Podstawowe oczka różniące się wysokością. |
| **Wbicie (punkt wbicia, insertion)** | Miejsce, w które wkłada się szydełko przy tworzeniu oczka, np. pod obie pętle, przednią lub tylną pętlę albo w łuk. |
| **Magiczne kółko (mr)** | Technika rozpoczęcia robótki okrężnej z zaciąganą pętlą. |
| **Okrążenie / rząd (warstwa)** | Kolejna warstwa oczek wykonywana po okręgu lub w linii. |
| **Dodawanie / łączenie (increase / decrease)** | Zwiększanie lub zmniejszanie liczby oczek w warstwie. |
| **Schemat (chart)** | Graficzny zapis wzoru za pomocą standardowych symboli. |
| **Próbka (swatch)** | Fragment robótki o znanej liczbie oczek, używany do kalibracji wymiarów. |

## 3.2 Model i algorytmy

| Termin | Definicja |
|---|---|
| **Graf wzoru** | Reprezentacja wzoru, w której wierzchołki to oczka, a krawędzie to relacje między nimi. |
| **Układ (layout)** | Przypisanie symbolom pozycji, obrotów i skal na płaszczyźnie. |
| **Algorytm force-directed** | Metoda wyznaczania układu przez symulację sił przyciągania i odpychania między wierzchołkami. |
| **Stress / stress majorization** | Miara i metoda minimalizacji rozbieżności między odległościami na rysunku a odległościami docelowymi. |
| **Proporcja H/W (aspect ratio)** | Stosunek wysokości do szerokości oczka, z wartością preferowaną i granicami. |
| **Kalibracja** | Dopasowanie parametrów geometrii do włóczki i napięcia użytkownika na podstawie próbki. |

### 3.3 Technologie i jakość

| Termin | Definicja |
|---|---|
| **SVG** | Wektorowy format grafiki, używany do renderowania i eksportu schematów. |
| **Undo/redo** | cofanie i ponawianie operacji. |
| **Web Worker** | Mechanizm przeglądarki do wykonywania obliczeń w osobnym wątku. |
| **MVP** | Minimalna wersja produktu realizująca podstawowe funkcje. |
| **NFR** | Wymaganie niefunkcjonalne, dotyczące jakości, a nie funkcji systemu. |
| **Percentyl (p95)** | Wartość, poniżej której mieści się 95% pomiarów. |

# 4 Zakres projektu: MVP i opcjonalne funkcjonalności

Zakres projektu został podzielony na MVP (Minimum Viable Product) - kluczowe funkcjonalności, które muszą działać, żeby móc uznać aplikację za funkcjonalną, oraz opcjonalne funkcjonalności, które poszerzają możliwości aplikacji, ale nie są niezbędne i mogą zostać zaimplementowane, jeśli starczy na to czasu.

## 4.1 Minimum Viable Product (MVP – Passing Scope)

1. **Graficzny interfejs edytora:** Pozwalający na łatwe i intuicyjne dodawanie, usuwanie i edycję elementów oraz wyświetlanie obecnego stanu wzoru.
2. **Automatyczne rozmieszczanie symboli:** Wyznaczenie wzajemnego położenia symboli odwzorowującego rzeczywiste ich rozmieszczenie po wyszydełkowaniu wzoru za pomocą force-directed algorytmu.
3. **Zapisanie i wczytanie postępów pracy:** Eksport i import danych schematu do pliku np. w formacie JSON, zachowującego informacje strukturalne.
4. **Eksport schematu do pliku w formacie graficznym:** Zapisanie gotowego schematu do pliku SVG, konwersja do PDF, PNG, JPEG.

## 4.2 Opcjonalne funkcjonalności

1. Możliwość ręcznych poprawek ustawienia poszczególnych symboli / grup symboli, w tym zablokowanie pozycji wybranych elementów.
2. Modyfikacja parametrów algorytmu automatycznego rozmieszczania.
3. Identyfikacja i wizualne zaznaczenie potencjalnie problematycznych fragmentów, mogących świadczyć o błędach we wzorze.
4. Konwersja schematu do instrukcji tekstowych i na odwrót.
5. Grupowanie elementów i możliwość dodawania/usuwania całych grup symboli.
6. Wizualizacja kolejności wykonywania elementów przy odtwarzaniu wzoru.

# 5 Wymagania funkcjonalne

Wymagania funkcjonalne zostały zebrane w formie historyjek użytkownika (User Stories) i przedstawione na diagramie przypadków użycia. Wymagania dotyczą tylko zakresu MVP, opcjonalne funkcjonalności nie zostały tutaj uwzględnione. W tym przypadku mamy tylko jednego użytkownika - osobę projektującą wzór.

### US-01: Dodawanie oczka przez kliknięcie

**Jako:** Osoba projektująca wzór,
**Chcę:** wybrać typ elementu z palety symboli i dodać go, klikając na element, w który chcę go wbić,
**Aby:** tworzyć wzór zgodnie z tym, jak powinien być on wyszydełkowany.

**Kryteria akceptacji (Gherkin):**

**Given:** wzór zawiera co najmniej jeden element, a w palecie wybrany jest słupek
**When:** użytkownik klika symbol na schemacie
**Then:** do wzoru zostaje dodany słupek wbity w kliknięty element, a na schemacie pojawia się nowy symbol.

**Scenariusz: niedozwolone wbicie**
**When:** wybrany jest typ elementu, którego nie można wbić we wskazane miejsce
**Then:** wzór pozostaje bez zmian i wyświetla się komunikat z przyczyną

### US-02: Usuwanie i edycja oczek

**Jako:** Osoba projektująca wzór,
**Chcę:** usuwać oczka i zmieniać ich właściwości,
**Aby:** poprawiać błędy bez budowania wzoru od nowa.

**Kryteria akceptacji (Gherkin):**

**Scenariusz: usunięcie ostatniego oczka**
**Given:** wzór zawiera co najmniej jedno oczko
**When:** użytkownik wybiera "Usuń ostatnie oczko"
**Then:** ostatnio dodane oczko zostaje usunięte razem z jego połączeniami i pozycja nici wraca do poprzedniego oczka

**Scenariusz: zmiana typu oczka**
**Given:** użytkownik zaznaczył oczko
**When:** wybiera inny typ oczka w panelu właściwości, np. słupek zamiast półsłupka
**Then:** oczko zmienia typ, a jego połączenia pozostają bez zmian i układ symboli zostaje przeliczony

### US-03: Dodawanie i łączenie oczek

**Jako:** Osoba projektująca wzór,
**Chcę:** dodawać kilka oczek w jedno miejsce i łączyć oczka,

**Aby:** zmieniać kształt robótki.

**Kryteria akceptacji (Gherkin):**

**Scenariusz: dodawanie oczek w jedno miejsce**
**Given:** w palecie wybrany jest typ oczka pozwalający na wbicie
**When:** użytkownik klika to samo oczko kilka razy z rzędu
**Then:** każde kliknięcie dodaje nowy słupek wbity w to samo oczko i symbole rozkładają się wachlarzowo nad oczkiem bazowym

**Scenariusz: łączenie kilku oczek**
**Given:** w palecie wybrany jest typ oczka pozwalający na wbicie, a tryb łączenia jest włączony
**When:** użytkownik zaznacza kolejno dwa lub więcej sąsiednich oczek i zatwierdza wybór
**Then:** zostaje dodany jeden półsłupek wbity we wszystkie zaznaczone oczka, a jego symbol odzwierciedla łączenie

### US-04: Zmiana okrążenia/rzędu

**Jako:** Osoba projektująca wzór,
**Chcę:** zakończyć okrążenie/rząd i rozpocząć następne,
**Aby:** budować wzór warstwami i zwiększyć jego czytelność.

**Kryteria akceptacji (Gherkin):**

**Given:** bieżąca warstwa ma numer $n$ i zawiera co najmniej jeden element
**When:** użytkownik wybiera opcję "nowa warstwa"
**Then:** numer bieżącej warstwy wynosi *n+1* i dodawane elementy mają przypisany numer warstwy *n+1*.

### US-05: Operacje undo/redo

**Jako:** Osoba projektująca wzór,
**Chcę:** cofać i ponawiać operacje,
**Aby:** swobodnie eksperymentować.

**Kryteria akceptacji (Gherkin):**

**Scenariusz: cofnięcie operacji**
**Given:** użytkownik wykonał operację modyfikującą wzór
**When:** naciska Ctrl+Z lub przycisk "Cofnij"
**Then:** wzór, pozycja nici i numer okrążenia wracają do stanu sprzed operacji

**Scenariusz: ponowienie operacji**
**Given:** użytkownik cofnął operację
**When:** naciska Ctrl+Y lub przycisk "Ponów"
**Then:** wzór wraca do stanu po tej operacji

### US-06: Przesuwanie i skalowanie wzoru

**Jako:** Osoba projektująca wzór,
**Chcę:** przesuwać i przybliżać schemat,
**Aby:** wygodnie pracować zarówno z detalami, jak i z całym wzorem.

**Kryteria akceptacji (Gherkin):**

**Scenariusz: przybliżanie i oddalanie**
**Given:** schemat jest wyświetlony
**When:** użytkownik obraca kółkiem myszy albo klika "+"lub "
**Then:** widok przybliża się lub oddala względem pozycji kursora albo środka widoku i aktualna skala jest widoczna w interfejsie

**Scenariusz: przesuwanie widoku**
**Given:** schemat jest wyświetlony
**When:** użytkownik przeciąga tło
**Then:** widok przesuwa się zgodnie z ruchem kursora

### US-07: Automatyczne rozmieszczanie symboli

**Jako:** Osoba projektująca wzór,
**Chcę:** aby symbole układały się automatycznie po wprowadzeniu zmian,
**Aby:** uniknąć konieczności ręcznego ustawiania i obracania symboli.

**Kryteria akceptacji (Gherkin):**

**Given:** wzór jest wyświetlony na schemacie
**When:** użytkownik modyfikuje wzór
**Then:** układ symboli zostaje przeliczony i wyświetlają się one w zaktualizowanych pozycjach

**Scenariusz: orientacja i proporcje symboli**
**Given:** układ został przeliczony
**When:** użytkownik ogląda schemat
**Then:** każdy symbol jest skierowany podstawą w stronę oczka, w które jest wbity, proporcje H/W każdego symbolu mieszczą się w granicach zdefiniowanych dla jego typu i symbole nie nakładają się na siebie (na tyle na ile to możliwe)

**Scenariusz: duża liczba oczek**
**Given:** wzór zawiera ok. 500 oczek
**When:** trwa przeliczanie układu
**Then:** interfejs reaguje na działania użytkownika (przewijanie, przybliżanie, wybór w palecie)

### US-08: Zapis i odczyt

**Jako:** Osoba projektująca wzór,
**Chcę:** zapisać projekt do pliku i ponownie go otworzyć,
**Aby:** kontynuować pracę później.

**Kryteria akceptacji (Gherkin):**

**Given:** użytkownik utworzył wzór
**When:** zapisuje projekt do pliku, a następnie go wczytuje
**Then:** wzór, pozycja nici i numer bieżącego okrążenia pozostają takie same jak przed zapisem

**Scenariusz: niepoprawny plik**
**Given:** w edytorze jest otwarty wzór
**When:** użytkownik wczytuje uszkodzony lub niepoprawny plik
**Then:** wyświetla się komunikat błędu, a bieżący wzór pozostaje niezmieniony

**Scenariusz: niezapisane zmiany**
**Given:** wzór ma niezapisane zmiany
**When:** użytkownik próbuje zamknąć lub odświeżyć stronę
**Then:** pojawia się ostrzeżenie o utracie zmian

### US-09: Eksport do pliku graficznego

**Jako:** Osoba projektująca wzór,
**Chcę:** wyeksportować schemat do pliku SVG, PDF lub PNG,
**Aby:** go wydrukować lub opublikować.

**Kryteria akceptacji (Gherkin):**

**Given:** wzór zawiera co najmniej jedno oczko
**When:** użytkownik wybiera eksport do wybranego formatu
**Then:** w razie potrzeby wzór jest konwertowany do wybranego formatu, zostaje pobrany plik z całym schematem, niezależnie od bieżącego przybliżenia widoku.

# 6 Mierzalne wymagania niefunkcjonalne (NFRs –Quantifiable Non-Functional Requirements)

Poniższe wymagania niefunkcjonalne opisują oczekiwaną jakość działania systemu. Każde z nich ma mierzalną wartość docelową i określoną metodę weryfikacji. Podane progi są wartościami wstępnymi i mogą zostać skorygowane na podstawie pomiarów prototypu.

| ID | Kategoria | Docelowa mierzalna metryka | Sposób weryfikacji |
|---|---|---|---|
| **NFR-P1** | **Wydajność** | Dodanie oczka jest widoczne na schemacie w $\leq$ 200 ms (p95) dla wzoru do 500 oczek. | Pomiar czasu od kliknięcia do renderu, np. `performance.now()` lub React Profiler |
| **NFR-P2** | **Wydajność** | Pełne przeliczenie układu dla wzoru 500 oczek trwa $\leq$ 2 s. | Benchmark na zestawie testowych wzorów |
| **NFR-P3** | **Wydajność** | Podczas przeliczania układu interfejs utrzymuje $\geq$ 30 FPS przy przewijaniu i przybliżaniu. | Chrome DevTools, zakładka Performance |
| **NFR-P4** | **Wydajność** | Pierwsze załadowanie aplikacji: Largest Contentful Paint $\leq$ 2,5 s. | Chrome DevTools Lighthouse |
| **NFR-L1** | **Jakość układu** | 95% symboli ma proporcje H/W w zdefiniowanych granicach. | Automatyczny test na zestawie wzorów |
| **NFR-L2** | **Jakość układu** | Pole nakładania się symboli wynosi $\leq$ 1% ich łącznego pola. | Automatyczny test geometryczny |
| **NFR-L3** | **Jakość układu** | Ten sam wzór daje ten sam układ (deterministyczność przy stałym seedzie). | Test porównujący dwa uruchomienia |
| **NFR-C1** | **Kompatybilność** | Aplikacja działa poprawnie (wszystkie scenariusze MVP przechodzą) w przeglądarkach desktopowych w wersjach nie starszych niż: Chrome 155, Edge 155, Firefox 157, Safari 26 (macOS). Stan wersji na dzień 19.10.2026. | Testy manualne lub Playwright na wielu przeglądarkach |
| **NFR-C2** | **Kompatybilność** | Interfejs jest używalny od rozdzielczości 1280×720. | Testy wizualne |

| | | | |
|---|---|---|---|
| **NFR-R1** | **Niezawodność** | Zapis i wczytanie projektu zachowuje 100% danych wzoru. | Test automatyczny: zapis, odczyt, porównanie |
| **NFR-R2** | **Niezawodność** | Niepoprawny plik projektu nigdy nie powoduje awarii aplikacji. | Testy z uszkodzonymi plikami (fuzzing JSON) |
| **NFR-R3** | **Niezawodność** | Niezapisany stan jest odtwarzany po przypadkowym odświeżeniu strony (autozapis co $\leq 30$ s). | Test manualny |
| **NFR-M1** | **Utrzymywalność** | Pokrycie testami modułu modelu i algorytmów $\geq 80\%$. | Raport pokrycia, np. Vitest z istanbul |

# 7 Zarządzanie ryzykiem – analiza SWOT

Analiza SWOT przedstawia mocne i słabe strony projektu (czynniki wewnętrzne) oraz szanse i zagrożenia wynikające z jego otoczenia (czynniki zewnętrzne).

| | **Pozytywne** | **Negatywne** |
|---|---|---|
| **Wewnętrzne** | **Mocne strony (S):**<br>- Nowoczesny, dobrze udokumentowany stos (React, TypeScript, SVG)<br>- Architektura z oddzielonym modelem, łatwa do testowania<br>- Nacisk na UX, którego brakuje istniejącym narzędziom | **Słabe strony (W):**<br>- Ograniczone doświadczenie z algorytmami layoutu grafów<br>- Niepewne dane o geometrii oczek<br>- Ograniczony czas i dwuosobowy zespół<br>- Trudność obiektywnej oceny "ładności"układu<br>- Zależność dwóch części pracy od wspólnego interfejsu |
| **Zewnętrzne** | **Szanse (O):**<br>- Niszowy obszar z małą liczbą narzędzi łączących automatyczny układ z edycją<br>- Duża społeczność szydełkująca i rynek wzorów<br>- Gotowe biblioteki (d3-force, WebCola) skracające implementację<br>- Możliwość dalszego rozwoju: instrukcje tekstowe, 3D, kalibracja<br>- Rosnące zainteresowanie badawcze cyfryzacją rękodzieła (prace z 2022–2025) | **Zagrożenia (T):**<br>- Istniejące narzędzia (CrochetPARADE, Stitch Fiddle) mogą rozwinąć podobne funkcje<br>- Brak standardu notacji utrudnia uniwersalność<br>- Ograniczony dostęp do użytkowników testowych<br>- Ryzyko, że wynik nie spełni oczekiwań estetycznych osób szydełkujących<br>- Zmiany wymagań ze strony promotora |

# 8 Harmonogram i macierz RACI

**[Tydzień 01-03] Etap 1:** Rozpisanie wymagań, Podział na MVP i funkcje opcjonalne, Research dostępnych narzędzi i technologii, Analiza ryzyka.
**[Tydzień 04] Etap 2:** Zdefiniowanie modeli, architektury, komunikacji między modułami, Wybór algorytmu(-ów) do rozmieszczania symboli.
**[Tydzień 05-06] Etap 3:** Implementacja poszczególnych komponentów MVP, Testy jednostkowe.
**[Tydzień 07-08] Etap 4:** Integracja modułów, Testy integracyjne, Pierwsza funkcjonalna wersja aplikacji, Prezentacja aplikacji promotorowi i zebranie informacji o ewentualnych koniecznych poprawkach.
**[Tydzień 09-11] Etap 5:** Bufor zapasowy czasu w razie potrzeby, Inkrementalna implementacja, testowanie i integracja dodatkowych funkcjonalności.
**[Tydzień 12] Zamrożenie kodu,** Zakończenie prac nad aplikacją.
**[Tydzień 13-14] Etap 6:** Prezentacja finalnej wersji aplikacji promotorowi, Ewaluacja wydajności, Pisanie pracy dyplomowej
**[Tydzień 15]** Uzyskanie oceny od promotora, Przygotowanie wersji do wysłania do APD, Przygotowanie prezentacji na obronę

## 8.1 Macierz odpowiedzialności (RACI)

Definicje ról:

- **R (Responsible):** Osoba wykonująca zadanie (bezpośredni autor kodu / dokumentacji).
- **A (Accountable):** Osoba ponosząca ostateczną odpowiedzialność za jakość wyników i dotrzymanie terminów (**dokładnie 1 osoba na każde zadanie!**).
- **C (Consulted):** Ekspert merytoryczny, z którym konsultowano kwestie techniczne i który dokonał przeglądu.
- **I (Informed):** Osoba na bieżąco informowana o postępach i ostatecznych decyzjach.

| **Zadanie** | **Anna** | **Wiktor** | **Promotor** |
|---|---|---|---|
| Cel, zakres, MVP, wymagania, harmonogram, zarządzanie ryzykiem (Etap 1) | **A** | R | C |
| Projekt architektury i interfejsów (Etap 2) | **A / R** | R | C |
| Przegląd literatury i dane o geometrii oczek (Etap 2) | **A / R** | C | I |
| Model danych i operacje na grafie (Etap 2) | **A / R** | C | C |
| Algorytm rozmieszczania i metryki estetyczne (Etap 2) | **A / R** | I | C |
| Projekt UX/UI (Etap 2) | C | **A / R** | I |
| Implementacja edytora (SVG, interakcje, undo/redo) (Etap 3) | C | **A / R** | - |
| Testy automatyczne i code review (Etap 3) | R | **A / R** | C |
| Integracja modułu algorytmicznego z edytorem (Etap 4) | **A / R** | R | - |
| Ewaluacja algorytmu (Etap 4) | **A / R** | C | C |

| | | | |
|---|---|---|---|
| Implementacja dodatkowych funkcjonalności (Etap 5) | R | **A / R** | C |
| Praca pisemna: wstęp, przegląd literatury, użyte algorytmy, model danych (Etap 6) | **A / R** | R | C |
| Praca pisemna: architektura rozwiązania, implementacja, wyniki (Etap 6) | R | **A / R** | C |
| Praca pisemna: Redakcja i zatwierdzenie całości (Etap 6) | R | R | **A** |